II. Related Work

Research on encrypted-traffic recognition primarily addresses traffic representation and unknown-class detection. Representation methods extract discriminative features through sequence modeling, graph learning, and multilevel byte representations. To address the continual emergence of unknown services and new attacks in operational networks, open-set detection methods increasingly use classification confidence, probability distributions, and distance measures. We review these two directions below.

A. Encrypted-Traffic Representation Learning

The widespread use of encryption in network communications limits traditional traffic identification methods that rely on port numbers and plaintext payloads. Existing studies use information that remains observable after encryption, including packet bytes, lengths, directions, and timing, and apply deep learning to extract latent behavioral features.

Byte-sequence representations constitute a major direction in encrypted-traffic classification. These methods typically convert raw packets into byte sequences and use convolutional neural networks or Transformers to extract local features and contextual dependencies. TrafficFormer [1] introduces traffic-specific pretraining, strengthens traffic representations through fine-grained classification tasks, and fine-tunes the model on downstream tasks. Such methods learn rich sequence semantics from raw bytes but do not directly model explicit interactions between packets.

Graph-based methods capture additional structural information by representing traffic as graphs and learning packet relationships with graph neural networks. GraphDApp [3] converts packet-length sequences into traffic interaction graphs for decentralized application identification. Related flow interaction graph research [2] constructs graphs from packet interactions to capture structural characteristics of communication directions and transmission processes. TFE-GNN [7] instead constructs traffic graphs using pointwise mutual information between bytes. Its graph neural network and feature fusion module extract fine-grained byte associations. These studies indicate that packet interactions and byte associations provide information complementary to conventional sequence representations.

Two-dimensional byte representations also support traffic feature extraction. YaTC [4] introduces a Multi-Level Flow Representation (MFR) that organizes packet headers and payloads into hierarchical matrices. It extracts features through packet-level and flow-level attention and uses a masked autoencoder for pretraining. This representation preserves within-packet byte organization and cross-packet hierarchical information, offering a feature perspective distinct from sequence modeling.

Existing representation methods thus capture byte context, packet interactions, and multilevel byte organization from different perspectives. These representations offer complementary information, while a single view cannot fully characterize both encrypted-traffic content and interaction behavior. Fusing heterogeneous representations into a unified feature space that incorporates multiple dimensions remains an important direction for improving traffic recognition.

B. Unknown-Traffic Detection Methods

Open-set traffic detection requires both accurate known-class recognition and identification of traffic from classes absent during training. This extends closed-set recognition, which classifies only the classes observed during training. Existing approaches primarily use discriminative confidence or data distributions and prototype-based measures.

1) Discriminative Unknown-Traffic Detection: Discriminative methods learn decision boundaries between known classes and use classification outputs to assess whether a test sample belongs to a known class. Maximum softmax probability (MSP) is a common unknown-detection baseline. It measures classification confidence using the largest predicted class probability and identifies low-confidence samples as unknown [10]. Energy-based methods compute energy scores from classifier logits to improve separation between known and out-of-distribution samples [11].

Studies targeting open network environments also introduce dedicated mechanisms for unknown-class discrimination. Trident [6] decomposes known-class detection into independent one-class learning tasks, supports fine-grained unknown-traffic recognition, and allows incremental class updates. RoNeTC [8] models classification uncertainty through multiview feature extraction and second-order classification probability estimation. It dynamically fuses predictions from different views to improve recognition reliability in open environments.

Discriminative methods generally provide a direct detection pipeline, but their detection performance depends partly on classifier confidence. Unknown traffic that receives high confidence within a known-class decision region may still be incorrectly accepted as known. Classification confidence alone therefore cannot fully characterize the actual difference between a test sample and known-class feature distributions.

2) Generative and Prototype-Based Unknown-Traffic Detection: Generative methods learn known-class data distributions or latent representations and detect unknown traffic through deviations from those distributions. Yang et al. [9] propose a two-stage detection method that combines a conditional variational autoencoder (CVAE) with extreme value theory (EVT). Class-conditional autoencoders first learn known-class features; reconstruction errors and their tail distributions then identify unknown attacks.

Open-Detect, proposed by Meng et al. [5], introduces Gaussian class prototypes. Generative constraints promote intraclass compactness, while discriminative constraints increase separation between classes. The method maps each known class to a Gaussian prototype in the latent space and detects unknown classes using the KL divergence between a sample's posterior distribution and the class prototypes. It thereby addresses both known-class classification and unknown-class rejection.

Probability-distribution and prototype-distance methods assess deviations from known classes in feature space, complementing classification-confidence methods. However, class-prototype measures primarily describe a sample's position

relative to the overall class distribution. A single prototype cannot adequately characterize uneven within-class distributions or changes in local sample density. Combining overall class structure with local neighborhoods of actual training samples can therefore provide additional feature evidence for unknown-class discrimination.

C. Summary

Prior work has advanced encrypted-traffic representation learning and unknown-class detection, while leaving room for further improvement. Traffic representations emphasize particular feature dimensions, and further study is needed to exploit complementary information across byte sequences, packet interactions, and multilevel byte structure. Existing unknown-detection methods primarily rely on classification confidence, probability distributions, or class prototypes. These methods have difficulty jointly characterizing support relative to the overall position of known classes and local neighborhoods of actual training samples.

III. Method

We propose an open-set encrypted-traffic recognition method that combines three-view representation learning with global and local empirical support. The method characterizes traffic through byte sequences, packet interactions, and multilevel byte organization, and constructs a unified representation through Gaussian adaptation and deterministic fusion. It then combines empirical class centroids with within-class sample neighborhoods to measure deviations from known classes in terms of both global position and local neighborhood relationships. The resulting rejection score integrates global and local information for known-class classification and unknown-class rejection. Fig. 3-1 presents the overall framework.

A. Problem Definition and Notation

In open-set encrypted-traffic recognition, the labeled training set contains N samples, $\mathcal{D}_{\mathrm{tr}} = \{(x_i, y_i)\}_{i=1}^{N}$. Here, $x_i \in \mathcal{X}$ denotes a bidirectional flow, $\mathcal{X}$ is the traffic sample space, and $y_i \in \mathcal{C}_K = \{1, \ldots, C\}$ is its class label, with C known classes. The known-class validation set is $\mathcal{D}_{\mathrm{val}}$, whose labels also belong to $\mathcal{C}_K$.

At test time, the model processes traffic from both the known-class set $\mathcal{C}_K$ and the unknown-class set $\mathcal{C}_U$. Unknown classes appear in neither the training set nor the validation set, and $\mathcal{C}_K \cap \mathcal{C}_U = \varnothing$. Test samples therefore extend beyond the classes observed during training.

Our objective is to learn an open-set recognition model f that maps an input flow to a known-class label or a unified unknown label:

$$f : \mathcal{X} \to \mathcal{C}_K \cup \{\perp\}, \tag{1}$$

Here, $\perp$ denotes the unknown label. The learning objective has two parts: (1) accurately distinguish flows in $\mathcal{C}_K$ and output their known-class labels; and (2) identify flows in $\mathcal{C}_U$ and assign them the unified unknown label. The model thus addresses both known-class classification and unknown-class rejection. Table 3-1 summarizes the main notation; other symbols are defined at first use.

TABLE 3-1
Main notation and definitions

Symbol	Definition
$\mathcal{C}_K, \mathcal{C}_U$	Known- and unknown-class sets
$\mathcal{D}_{\mathrm{tr}}, \mathcal{D}_{\mathrm{val}}$	Known training and validation sets
$z_v(x), \mu_v(x)$	Encoded view features and latent mean after Gaussian adaptation
$h(x), \ell(x)$	Fused features and known-class classification score vector
$\mathcal{T}_c, b_c$	Training representation set and empirical centroid of class c
$c^*(x)$	Class of the nearest centroid
$d_G(x), d_L(x)$	Global centroid distance and local nearest-neighbor distance
$s(x), \tau$	Combined rejection score and rejection threshold

B. Three-View Traffic Representation Learning

Encrypted-traffic class characteristics appear in byte combinations and contextual relationships, interactions between packets, and the internal hierarchical structure of packets. We construct byte-sequence, flow-interaction-graph, and byte-image views to learn byte-context dependencies, packet interactions, and multilevel byte organization, respectively. Let $x = \{p_1, p_2, \ldots, p_{n_x}\}$ denote the chronologically ordered packet sequence of a bidirectional flow, where n_x is the packet count. The sending direction of the first packet defines the forward direction. Each branch constructs its input from the same flow and extracts complementary flow-level features for subsequent Gaussian adaptation and fusion.

1) Byte-Sequence Branch: The byte-sequence branch captures local byte-combination patterns and contextual dependencies along the packet-byte sequence. We use pretrained TrafficFormer [1] as the byte-sequence encoder to map raw packet bytes to a flow-level sequence representation.

For the chronologically ordered flow $x = \{p_1, p_2, \ldots, p_{n_x}\}$, we select the first $\min(5, n_x)$ packets. After removing the Ethernet header, we retain at most 64 bytes per packet and form hexadecimal byte bigrams following the TrafficFormer input representation. For a byte sequence $(b_1, b_2, \ldots, b_m)$, the bigram representation is $(b_1 \| b_2, b_2 \| b_3, \ldots, b_{m-1} \| b_m)$, where $\|$ denotes concatenation of adjacent bytes. TrafficFormer's token mapping converts these combinations into discrete tokens, which form a flow-level sequence in the original packet order. After adding the [CLS] token, we truncate or pad the input to a maximum of 320 tokens.

Let $T(x)$ denote the resulting token sequence. We extract features using the pretrained TrafficFormer encoder and fine-tune it with supervision on the known-class training set. The hidden representation at the [CLS] position in the final encoder layer serves as the flow-level byte-sequence feature:

$$z_t(x) = E_{\mathrm{TF}}(T(x))_{\mathrm{CLS}} \in \mathbb{R}^{768}, \tag{2}$$

Here, E_{TF} is the fine-tuned TrafficFormer encoder, and $z_t(x)$ is the 768-dimensional byte-sequence representation used by the subsequent multiview adaptation and fusion module.

2) Flow-Interaction-Graph Branch: The flow-interaction-graph branch characterizes communication behavior through relationships between packets, complementing the low-level

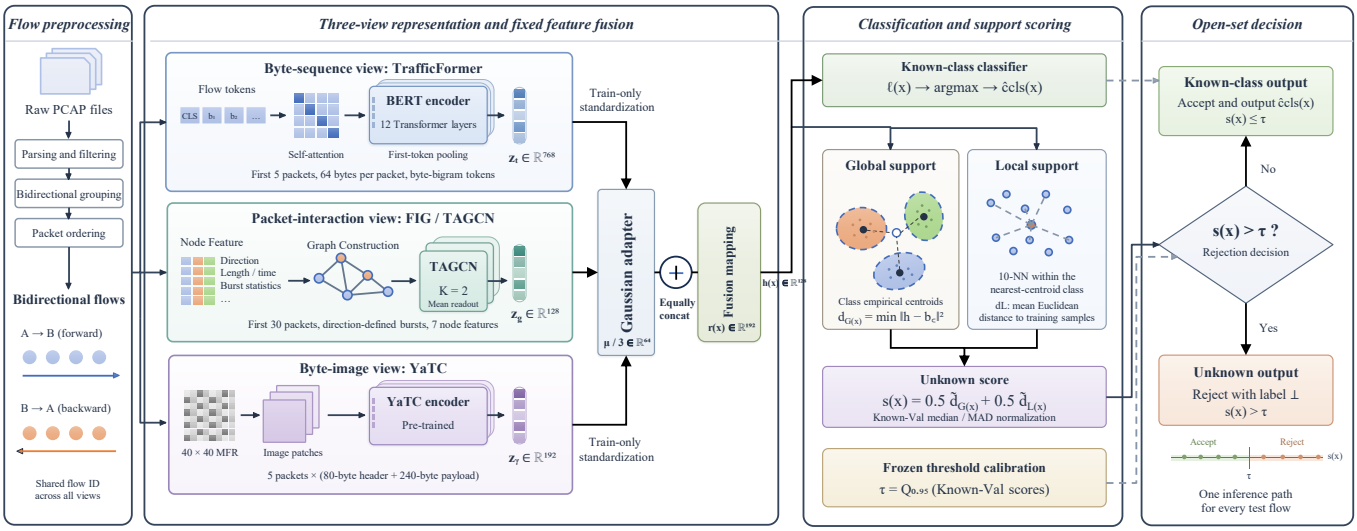


Fig. 3-1. Overall framework of the proposed method.

content features of the byte-sequence branch. Following existing Flow Interaction Graph (FIG) modeling approaches [2,3], we represent a bidirectional flow as a packet-level graph. A Topology Adaptive Graph Convolutional Network (TAGCN) learns a structured interaction representation that captures transmission direction, timing, and interaction topology.

For the chronologically ordered flow $x = \{p_1, p_2, \ldots, p_{n_x}\}$, we construct a graph from the first $\min(30, n_x)$ packets, with one node per packet. Consecutive packets with the same sending direction form a burst. Within each burst, edges connect consecutive packets in temporal order. Between adjacent bursts, edges connect their first nodes and their last nodes, capturing transmission continuity and direction changes. This procedure produces an undirected flow interaction graph $G(x) = (V, E)$, where V and E denote the node and edge sets. Fig. 3-2 illustrates burst segmentation, graph construction, node features, and neighborhood aggregation.

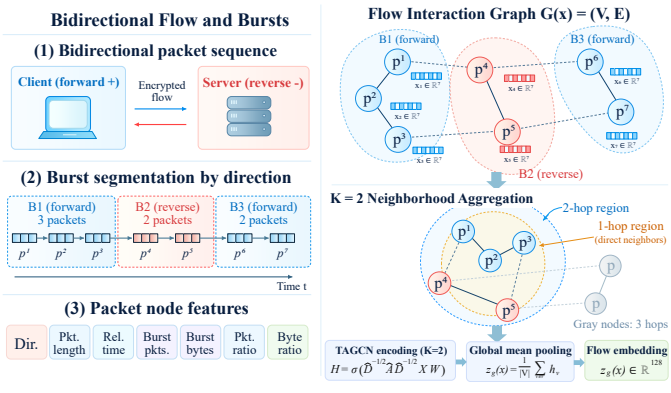


Fig. 3-2. Flow interaction graph construction and neighborhood aggregation.

Each node has seven interaction features: sending direction, packet length, elapsed time relative to the first packet, the packet count and total byte count of its burst, and the ratios of the current burst's packet and byte counts to those of the preceding burst. These features describe transmission attributes, local interaction scale, and changes between transmission stages. We collect packet nodes from flows in the known-class training set and compute the mean and standard deviation of each feature dimension. Dimension-wise standardization yields the node-feature matrix $X \in \mathbb{R}^{|V|\times 7}$. Validation and testing reuse the training statistics.

TAGCN aggregates neighborhoods of multiple orders to learn structural dependencies between packets. Let A denote the adjacency matrix, with normalized form $\hat{A} = D^{-1/2}(A + I)D^{-1/2}$, where D is the degree matrix of $A + I$. The node-feature update is

$$H = \mathrm{ReLU}\left(\sum_{k=0}^{2} \hat{A}^k X W_k + b\right), \tag{3}$$

Here, W_k and b are learnable parameters, and $H \in \mathbb{R}^{|V|\times 128}$ combines node attributes and multihop topology. Global mean pooling then produces the flow-level graph feature:

$$z_g(x) = \frac{1}{|V|} \sum_{u\in V} H[u,:] \in \mathbb{R}^{128}, \tag{4}$$

The resulting 128-dimensional representation $z_g(x)$ serves as input to multiview adaptation and fusion.

3) Byte-Image Branch: Packet headers and payloads exhibit hierarchical byte organization beyond byte-sequence patterns and packet interactions. Direct concatenation of raw bytes can allocate excessive representation space to longer packets and discard structural information from other packets. Following YaTC's Multi-Level Flow Representation (MFR) [4], we organize raw bytes into a two-dimensional matrix with a fixed layout. Hierarchical encoding captures byte-, packet-, and flow-level structure.

For a bidirectional flow x, we select the first five packets and retain an 80-byte header region and a 240-byte payload region per packet, padding missing content with zeros. The

fixed header and payload positions form an 8×40 packet-level matrix. Stacking these matrices in chronological order yields the 40×40 flow-level MFR matrix:

$$B(x) = \mathrm{Stack}(M_1, M_2, \ldots, M_5) \in \mathbb{R}^{40\times 40}, \tag{5}$$

Here, M_i is the two-dimensional byte matrix of packet i, containing separate header and payload regions. Fixed spatial positions for bytes at different levels preserve within-packet structure and flow-level organization. We normalize byte values to $[-1, 1]$ before encoding.

The pretrained YaTC encoder partitions the MFR matrix into nonoverlapping patches and constructs initial features through linear projection and positional embeddings. Packet-level attention first extracts byte relationships within packets, and row pooling (RP) aggregates them into row-level representations:

$$U_p = E_{\mathrm{PL}}(B(x)), \qquad U_r = \mathrm{RP}(U_p), \tag{6}$$

Here, E_{PL} comprises patch embedding and packet-level attention, U_p is the packet-level representation, and U_r is the representation after row pooling.

Flow-level attention then models dependencies across packets, and column pooling (CP) produces the flow-level representation:

$$U_f = E_{\mathrm{FL}}(U_r), \qquad z_y(x) = \mathrm{CP}(U_f) \in \mathbb{R}^{192}, \tag{7}$$

Here, E_{FL} is the flow-level attention encoder, U_f is its output, and $z_y(x)$ is the final 192-dimensional byte-image feature. We initialize the encoder with pretrained YaTC parameters and fine-tune it with supervision on the known-class training set. The resulting multilevel byte features feed the adaptation and fusion module.

C. Gaussian Adaptation and Deterministic Feature Fusion

The three views capture byte context, packet interactions, and multilevel byte organization, but their feature dimensions and numerical scales differ. We first standardize each encoded view feature $z_v(x)$ using known-training-set statistics, then map it into a common latent space with a view-specific Gaussian adapter. Let $v \in \{t, g, y\}$ index the byte-sequence, flow-interaction-graph, and byte-image views, respectively. The conditional distribution of each latent representation is

$$q_v(u \mid x) = \mathcal{N}\left(\mu_v(x), \mathrm{diag}\left(\exp(a_v(x))\right)\right), \tag{8}$$

Here, $\mu_v(x)$ and $a_v(x)$ denote the latent mean and log variance, respectively. All views share latent dimension d_u.

Gaussian adaptation jointly optimizes class discrimination and latent-distribution constraints. During training, reparameterized sampling from $q_v(u \mid x)$ produces latent representations. Classification cross-entropy preserves information relevant to known classes, while KL divergence from the standard normal distribution regularizes the latent distribution. The adaptation objective is

$$\begin{aligned}\mathcal{L}_{\mathrm{adapt}} = \frac{1}{3M}\sum_{i=1}^{M}\sum_{v\in\{t,g,y\}}\Big[&\mathcal{L}_{iv}^{\mathrm{CE}} \\ &+ \frac{\beta}{d_u} D_{\mathrm{KL}}\left(q_v(u \mid x_i) \,\|\, \mathcal{N}(0, I)\right)\Big].\end{aligned} \tag{9}$$

Here, M is the batch size. For sample i and view v, $\mathcal{L}_{iv}^{\mathrm{CE}}$ is the classification cross-entropy between the true label and the prediction of a view-specific classifier operating on the reparameterized latent sample. The coefficient β controls distribution regularization. The classification term preserves class differences, while the KL term constrains deviations from the standard normal distribution.

To eliminate randomness from latent sampling in distance calculations, we construct a deterministic fused representation from the latent means. We scale all three means by the same coefficient, concatenate them, and apply a learnable mapping to obtain fused features. A classification head outputs known-class scores:

$$\begin{aligned} r(x) &= \mathrm{concat}\left(\frac{\mu_t(x)}{3}, \frac{\mu_g(x)}{3}, \frac{\mu_y(x)}{3}\right), \\ h(x) &= F_\varphi(r(x)) \in \mathbb{R}^{128}, \\ \ell(x) &= g_\psi(h(x)) \in \mathbb{R}^{C}. \end{aligned} \tag{10}$$

Here, $r(x)$ is the concatenated representation, F_φ learns combinations of features across views, and g_ψ is the known-class classification head. The score $\ell_c(x)$ corresponds to class c. Global and local empirical support modeling also uses $h(x)$, placing known-class classification and unknown-class discrimination in the same representation space.

D. Global and Local Empirical Support Modeling

Known-class classification distinguishes classes, whereas unknown-class discrimination must also assess a test sample's relationship to known training samples. Class centroids summarize the overall location of training representations but cannot fully characterize within-class distributions. A sample near a centroid may still lie far from actual training samples. We therefore combine class centroids and within-class nearest neighbors in the fused space to characterize deviations in global position and local proximity. In the learned, fixed representation space, known training samples define a reference set for each class:

$$\begin{aligned}\mathcal{T}_c &= \{h(x_i) : y_i = c, (x_i, y_i) \in \mathcal{D}_{\mathrm{tr}}\}, \\ N_c &= |\mathcal{T}_c|.\end{aligned}$$

Here, $\mathcal{T}_c$ is the training representation set of known class c, and N_c is its size. The mean representation b_c serves as the empirical centroid, summarizing the class's overall position. For a test flow x, distances to all class centroids determine the nearest-centroid class $c^*(x)$. The corresponding squared Euclidean distance defines the global distance $d_G(x)$:

$$\begin{aligned} b_c &= \frac{1}{N_c}\sum_{h_i\in\mathcal{T}_c} h_i, \\ c^*(x) &= \arg\min_{c\in\mathcal{C}_K} \|h(x) - b_c\|_2^2, \\ d_G(x) &= \|h(x) - b_{c^*(x)}\|_2^2. \end{aligned} \tag{11}$$

The global distance measures deviation from the nearest class centroid. To characterize proximity to actual training representations, we select the k representations nearest to $h(x)$ in Euclidean distance from $\mathcal{T}_{c^*(x)}$. These representations form

the local neighborhood $\mathcal{N}_k(x)$. The local distance is their mean distance to the test representation:

$$d_L(x) = \frac{1}{k} \sum_{h_j \in \mathcal{N}_k(x)} \|h(x) - h_j\|_2, \qquad \mathcal{N}_k(x) \subseteq \mathcal{T}_{c^*(x)}. \tag{12}$$

A larger $d_L(x)$ indicates greater separation from the selected class's training neighborhood and thus weaker local empirical support. Global and local distances provide complementary geometric evidence of deviation from the class's overall location and actual training neighborhood. The nearest-centroid class $c^*(x)$ selects the class for neighborhood construction. The classification head still determines the final known-class label of an accepted sample; the two class assignments need not agree.

E. Robust Standardization and Open-Set Decision

The global distance $d_G(x)$ is a squared Euclidean distance to a centroid, whereas $d_L(x)$ is a mean Euclidean distance to within-class neighbors. Their numerical scales differ, so direct combination can allow one distance to dominate. We robustly standardize them separately using the known-class validation set. For $a \in \{G, L\}$, let $\mathcal{S}_a = \{d_a(x) : x \in \mathcal{D}_{\text{val}}\}$. Its median m_a and median absolute deviation MAD_a describe location and dispersion, yielding

$$\begin{aligned} m_a &= \text{median}(\mathcal{S}_a), \\ \text{MAD}_a &= \text{median}_{u \in \mathcal{S}_a} |u - m_a|, \\ \tilde{d}_a(x) &= \frac{d_a(x) - m_a}{\text{MAD}_a + \varepsilon}. \end{aligned} \tag{13}$$

Here, the positive constant $\varepsilon > 0$ prevents division by zero. Fixed equal weights combine the standardized distances into the rejection score $s(x)$. The empirical 95% quantile of known-validation scores determines the threshold τ:

$$\begin{aligned} s(x) &= \frac{1}{2}\tilde{d}_G(x) + \frac{1}{2}\tilde{d}_L(x), \\ \tau &= Q_{0.95}\left(\{s(x) : x \in \mathcal{D}_{\text{val}}\}\right). \end{aligned} \tag{14}$$

Here, $Q_{0.95}$ denotes the empirical quantile operator. Both the standardization statistics and the rejection threshold come from the known-class validation set. A higher $s(x)$ indicates greater combined deviation from known-class centroids and training neighborhoods. Scores above the threshold trigger an unknown decision. Otherwise, the classification head predicts $\hat{c}_{\text{cls}}(x) = \arg\max_{c \in \mathcal{C}_K} \ell_c(x)$. The final open-set decision rule is

$$f(x) = \begin{cases} \hat{c}_{\text{cls}}(x), & s(x) \le \tau, \\ \perp, & s(x) > \tau, \end{cases} \tag{15}$$

Here, $\perp$ is the unified unknown-class label.